

Mint-to-Burn Protocol – Phase 1: Core Framework

1. Objective Clarification

-  Purpose: Permanently destroy (burn) digital access or value after a verified, single-use event (e.g., ticket scan, voucher redemption, vote casting, etc.).
 -  Use Cases:
 - Ticket scalping prevention (concerts/events)
 - Voter fraud prevention (ballot burning post-validation)
 - Single-use coupons or vouchers
 - Secure one-time data access
-

Phase 2: Logic Design

1. Core Functions

- `mintItem(data, userInfo)`
- `validateBurn(tokenId, scanner/validation)`
- `burnToken(tokenId)`

2. Optional Layers

- Assign metadata (e.g., QR, user ID, biometric flag)
 - Track validation location/time
 - Temporary storage before burn (Mint-to-Limbo layer)
-

Phase 3: Technical Stack (No-Code to Code)

MVP (Proof of Concept) Options

Component	No-Code Tool / Tech
Frontend	Glide, Adalo, Bubble
Token System	TokenScript, web3mock, Airtable
Scanner Integration	QR/Bluetooth scanner + Zapier webhook
Backend Logic	Make.com or Xano
Burn Logging	Google Sheets, Firestore, or Supabase

Phase 4: Mint-to-Burn Flow

pgsql

CopyEdit

1. Admin mints token/item >
2. Token is assigned to user ID >
3. User presents token at event >
4. Scanner reads + verifies >
5. If valid: token is immediately "burned" (deleted/archived) >
6. Log is saved (burn hash, timestamp, validator ID)

Optional:

-  If invalid: notify admin or flag suspicious activity
-

Phase 5: Low-Cost Hardware Scanner Integration

- Barcode/QR scanner with USB/Bluetooth
 - Works like a keyboard input → scanned code auto-fills input field
 - No app required on scanner side
-

Phase 6: Start Prototyping

Would you prefer to start:

- A. With a **No-Code prototype** first (to demo and test flow)?
- B. Jump straight into building a **coded version** (Node.js + Firebase or Solidity + Web3.js)?
- C. Design the **UI/UX prototype** in Figma or Framer to visualize?